

Automated Server Deployment Using AWS CloudFormation

Aim

To automate the deployment of a web server infrastructure on AWS using CloudFormation templates, host a static web application, and make it publicly accessible through an EC2 instance configured with Nginx.

Objectives

- Automate AWS infrastructure provisioning using CloudFormation.
- Create an Amazon S3 bucket for storing website assets.
- Configure the S3 bucket with public read permissions for web content.
- Upload static website files (HTML, CSS, and JavaScript) to the S3 bucket.
- Provision networking components including a VPC, subnet, internet gateway, route table, and security group.
- Launch an EC2 instance with a public IP address.
- Automatically install and configure Nginx on the EC2 instance.
- Deploy the website by replacing the default Nginx content with files retrieved from Amazon S3.
- Verify successful deployment by accessing the application through the EC2 public IP address.

Project Overview

This project demonstrates Infrastructure as Code (IaC) using AWS CloudFormation to automate the deployment of a web server environment. Two CloudFormation templates are used:

1. An S3 template to create a publicly accessible bucket for website assets.
2. An infrastructure template to provision networking resources and an Ubuntu EC2 instance with Nginx installed and configured automatically.

The EC2 instance downloads the website files from the S3 bucket during initialization and serves them using Nginx, enabling users to access the application through the server's public IP.

Technologies Used

- Amazon Web Services (AWS)
- AWS CloudFormation
- Amazon EC2
- Amazon S3
- Amazon VPC
- Internet Gateway
- Route Tables
- Security Groups
- Nginx Web Server
- Ubuntu Linux
- HTML
- CSS
- JavaScript

Implementation Steps

Prepare the Website Files

Create or collect the static website assets including:

Automated Server Deployment Using AWS CloudFormation

- index.html
- CSS files
- JavaScript files
- Images and other supporting resources

These files constitute the web application to be deployed.

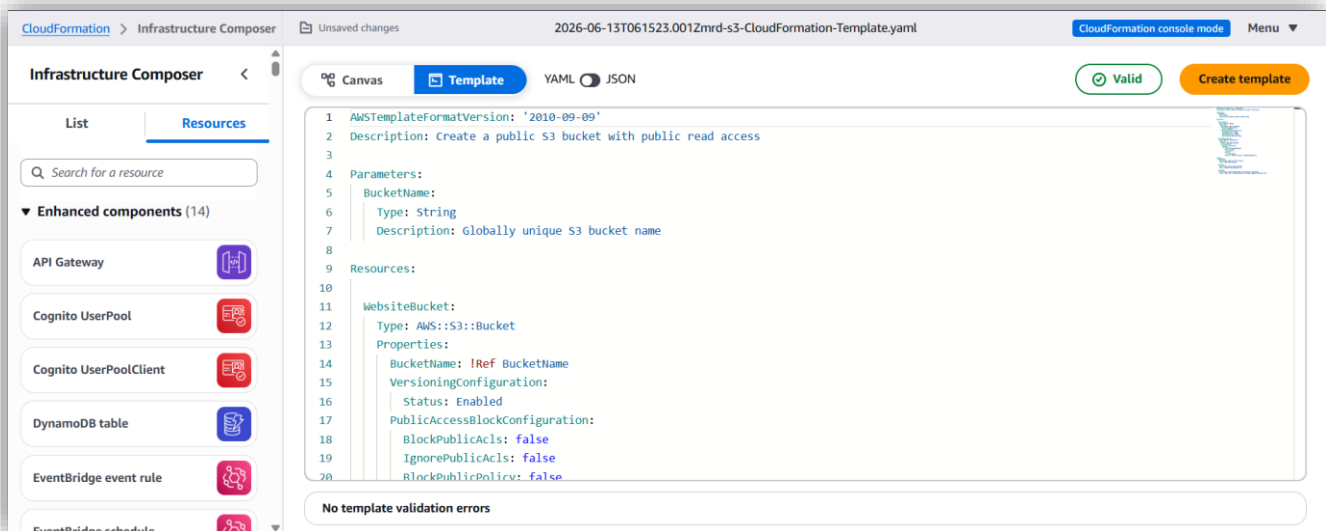
Earlier this week				
 index	11-06-2026 15:01	Chrome HTML Do...	25 KB	
 script	11-06-2026 15:01	JavaScript Source ...	25 KB	
 style.css	11-06-2026 15:01	CSSfile	34 KB	

Create an S3 Bucket Using CloudFormation

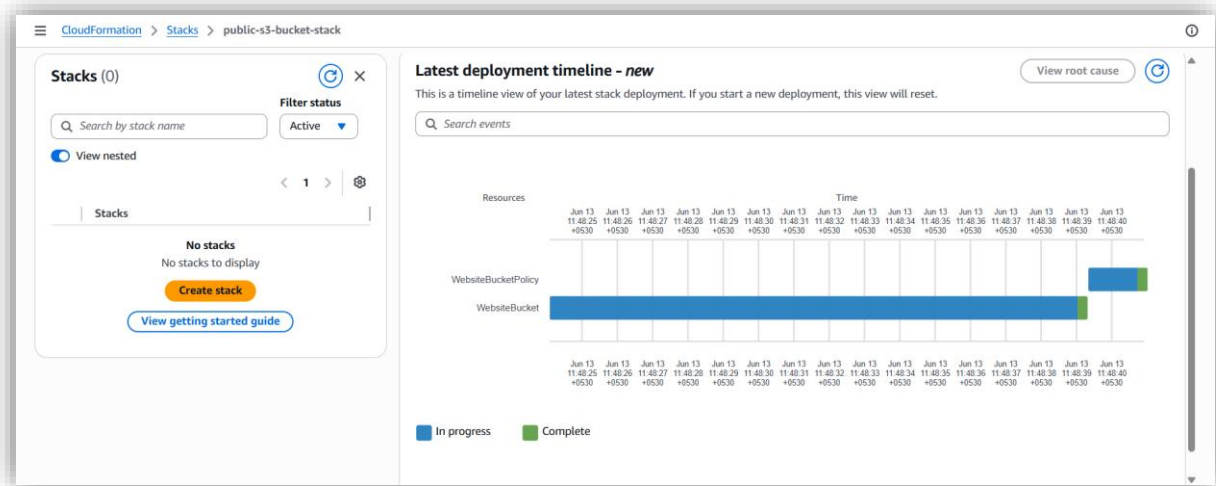
Deploy the S3 CloudFormation template to automatically create an Amazon S3 bucket.

Key configurations include:

- Globally unique bucket name
- Versioning enabled
- Public access settings configured to permit website asset access
- Bucket policy allowing public read access



Automated Server Deployment Using AWS CloudFormation



Verify S3 Bucket Creation

After successful deployment, confirm that the bucket has been created in the AWS Console.

Review:

- Bucket properties
- Permissions
- Public access configuration
- Bucket policy

The screenshot shows the AWS CloudFormation console for the 'public-s3-bucket-stack'. The 'Stacks (1)' section shows a single stack with the status 'CREATE_COMPLETE'. The 'Stack actions' menu is visible, with options like 'Delete stack', 'Update stack', and 'Stack actions'. The 'Create stack' button is also present.

Stack name	Status	Created time	Description
public-s3-bucket-stack	CREATE_COMPLETE	2026-06-13 11:48:22 UTC+0530	Create a public S3 bucket with public read access

Block public access (bucket settings)

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

Block all public access

Off

► Individual Block Public Access settings for this bucket

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::gowtham-webapp-bucket-2026/*"
    }
  ]
}
```

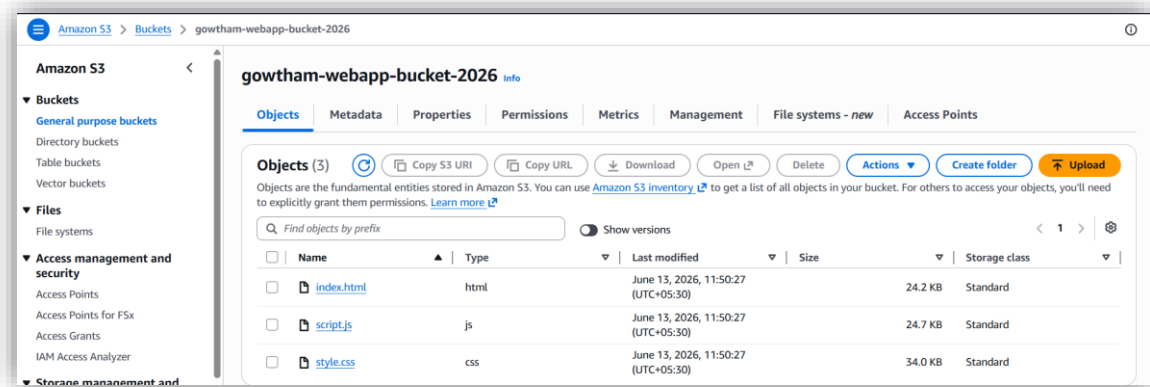
Automated Server Deployment Using AWS CloudFormation

Upload Website Files

Upload all static website files into the S3 bucket, including:

- HTML files
- CSS files
- JavaScript files
- Images and other assets

Ensure the uploaded objects are accessible for retrieval by the EC2 instance.

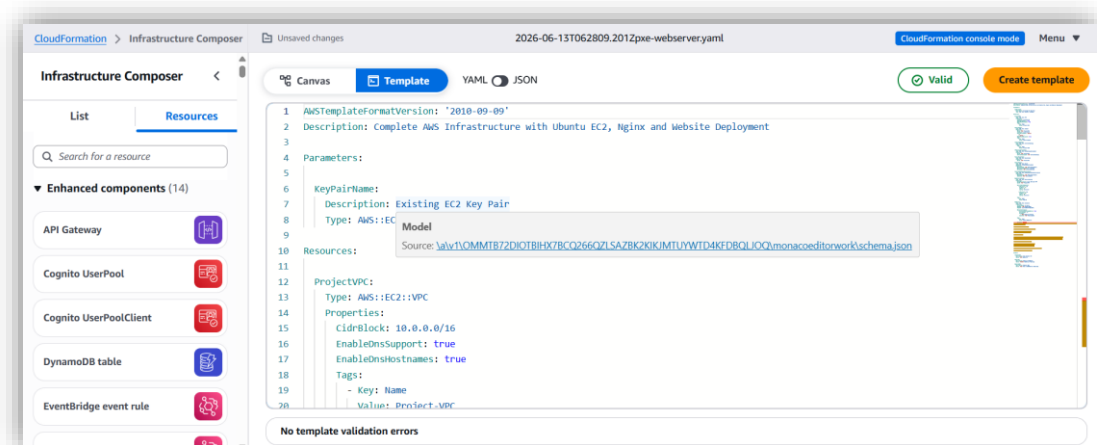


Deploy Infrastructure Using CloudFormation

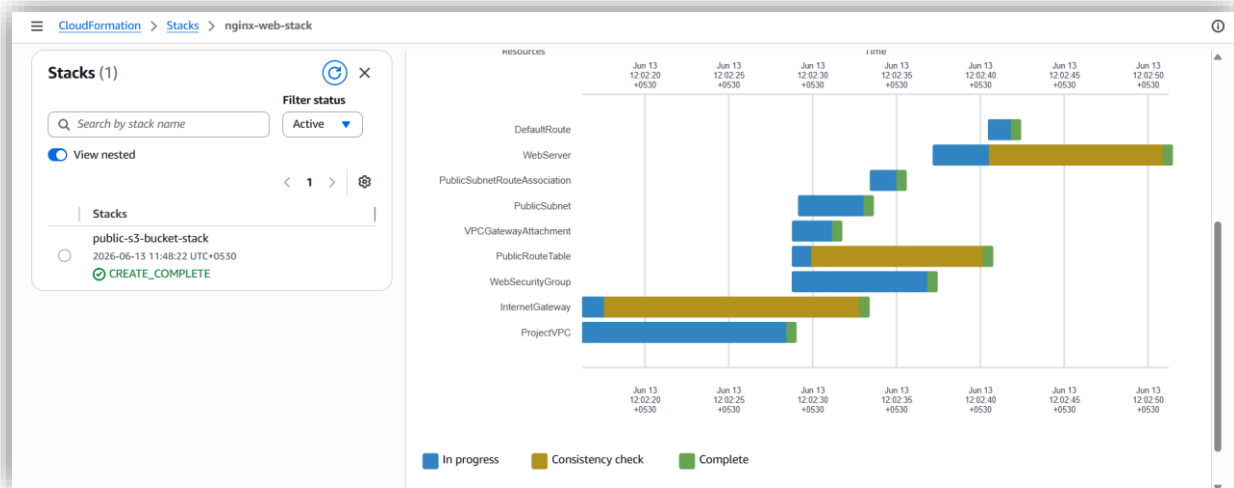
Launch the infrastructure stack using the second CloudFormation template.

The template provisions:

- Custom VPC
- Public subnet
- Internet Gateway
- Route table and associations
- Security Group permitting HTTP and SSH access
- Ubuntu EC2 instance with a public IP
- User Data script for automated server configuration



Automated Server Deployment Using AWS CloudFormation



Automatic EC2 Configuration

When the EC2 instance starts, the initialization script executes automatically to:

1. Update the operating system packages.
2. Install Nginx.
3. Download website files from the S3 bucket.
4. Replace the default Nginx web content.
5. Start and enable the Nginx service.

This eliminates manual server configuration and ensures repeatable deployments.

```
UserData: !Base64
Fn::Sub: |
  #!/bin/bash

  apt-get update -y

  apt-get install nginx wget -y

  systemctl enable nginx
  systemctl start nginx

  cd /tmp

  wget https://gowtham-webapp-bucket-2026.s3.ap-south-1.amazonaws.com/index.html
  wget https://gowtham-webapp-bucket-2026.s3.ap-south-1.amazonaws.com/style.css
  wget https://gowtham-webapp-bucket-2026.s3.ap-south-1.amazonaws.com/script.js

  rm -rf /var/www/html/*

  cp index.html /var/www/html/
  cp style.css /var/www/html/
  cp script.js /var/www/html/

  chown -R www-data:www-data /var/www/html
```

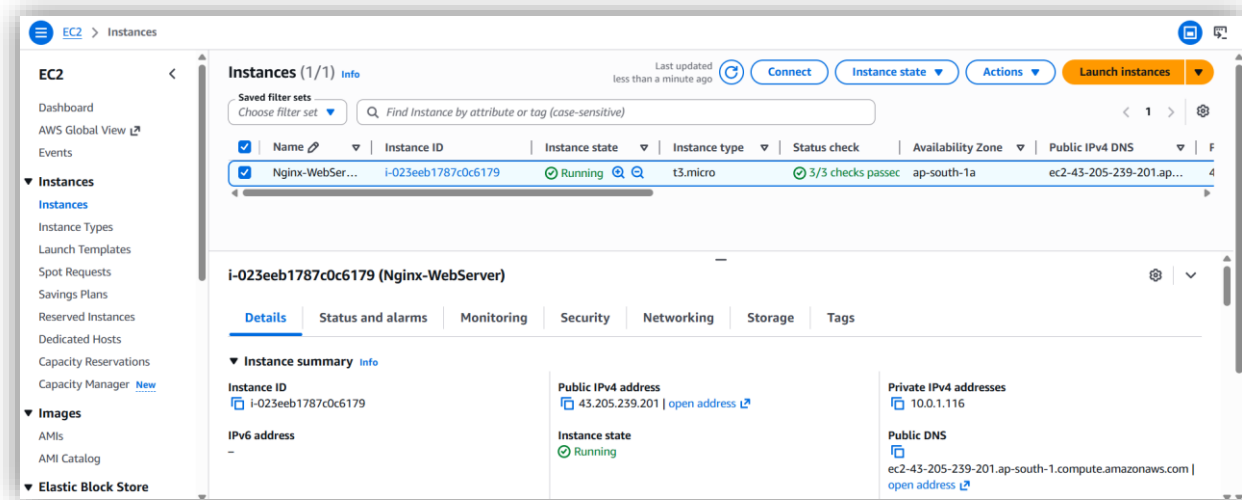
Automated Server Deployment Using AWS CloudFormation

Verify Infrastructure Resources

Confirm that AWS resources have been created successfully, including:

- VPC
- Public subnet
- Internet Gateway
- Route table
- Security Group
- EC2 instance
- Elastic networking configuration

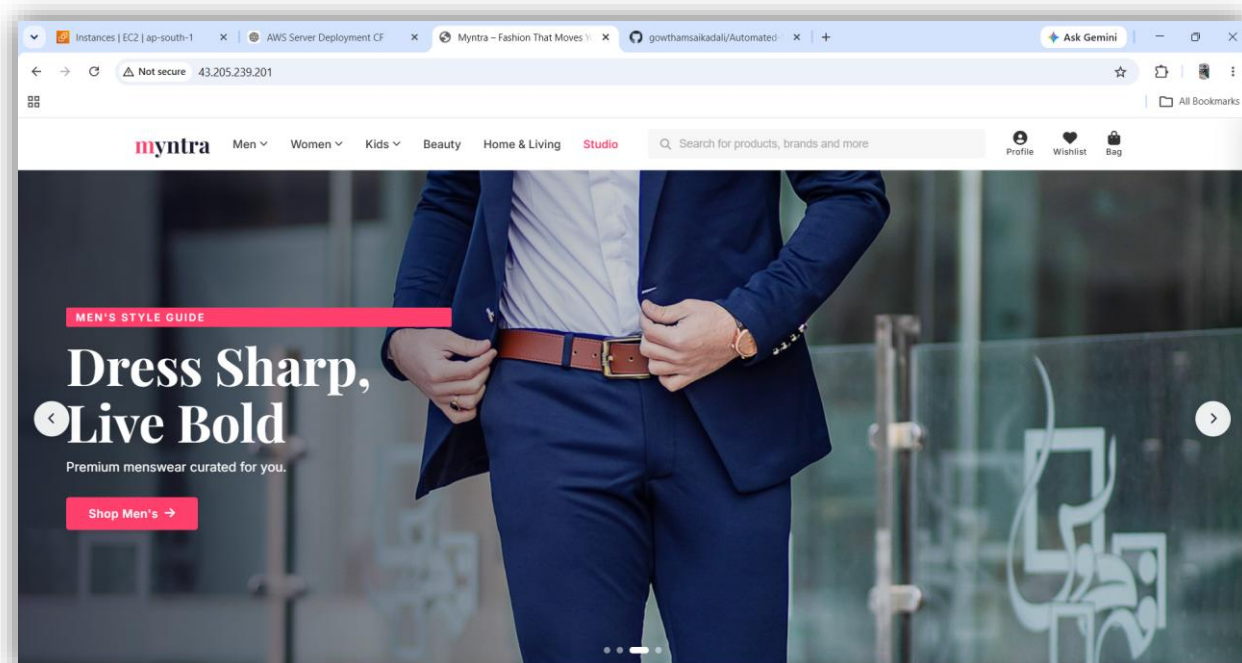
Verify that the EC2 instance is in the Running state and has a public IPv4 address.



Access the Deployed Website

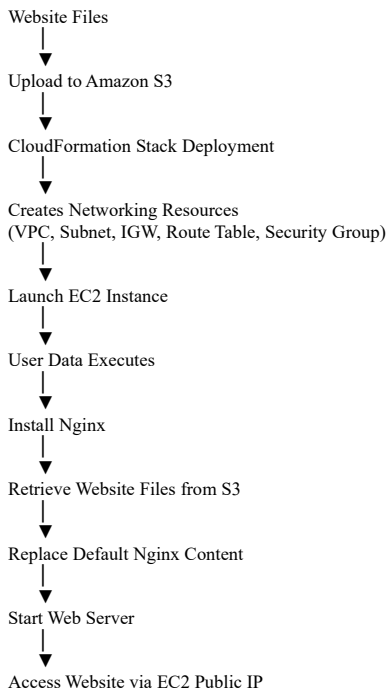
Copy the EC2 public IP address and open it in a web browser.

The Nginx server should serve the uploaded website instead of the default welcome page, demonstrating that the deployment completed successfully.



Automated Server Deployment Using AWS CloudFormation

Automation Workflow



Outcome

The project successfully automated the deployment of a web server using AWS CloudFormation. Infrastructure provisioning, server configuration, and website deployment were performed without manual intervention after stack creation. The static website was successfully hosted on an EC2 instance running Nginx and made publicly accessible through the instance's public IP address.

Benefits

- Infrastructure as Code (IaC) implementation
- Reduced manual configuration effort
- Consistent and repeatable deployments
- Faster provisioning process
- Simplified environment recreation
- Improved scalability and maintainability

Conclusion

This project demonstrates how AWS CloudFormation can be used to automate end-to-end server deployment and website hosting. By combining Amazon S3 for website asset storage with CloudFormation for infrastructure provisioning and EC2 User Data for automated server configuration, the deployment process becomes efficient, repeatable, and reliable. The final application is successfully served through an Nginx web server running on an EC2 instance and is accessible via its public IP address, showcasing the practical application of Infrastructure as Code principles in cloud environments.